Inside the first Mini Motorways model

A visual explanation of the model, its learning process and the first local experiment. Prepared for Mohammad on 28 October 2026.

The pipeline works. The strategy does not yet.
The model trained and its checkpoints reload, but all held-out evaluation scores remain zero. The 1,000-trip objective is still a research goal.

7 min 11 sec	242 games	7 trips
Learner runtime	Completed training episodes	Best observed training score

What this model is

A small neural network that chooses construction commands in the **OriginalReference** simulator. It receives structured numerical game state and learns through trial and error using **Maskable PPO**. It has 450,610 trainable parameters and runs on the CPU.

OBSERVE
Current city and materials

→

DECIDE
Tool, cells and upgrade

→

LEARN
Compare outcomes to estimates

Read the report

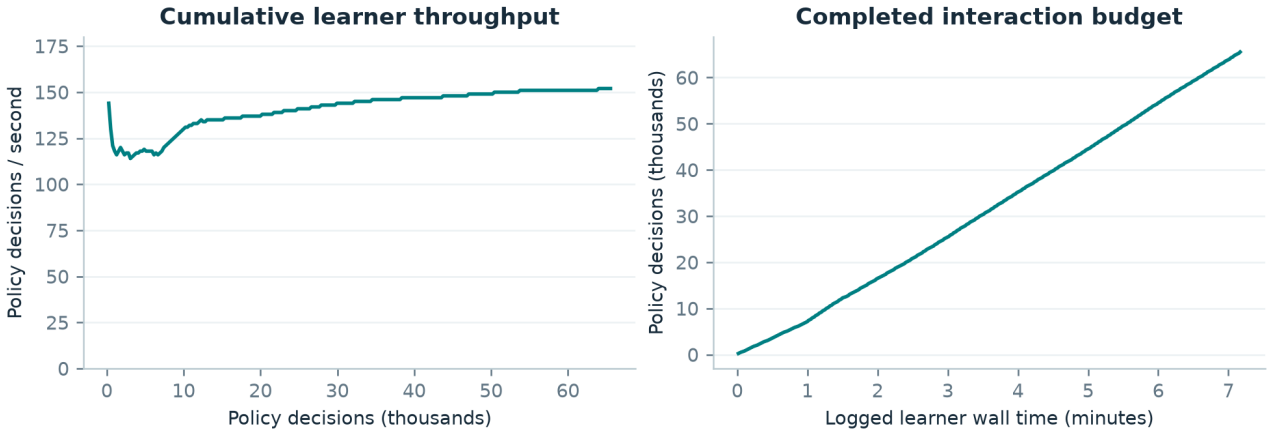
Topic	Pages
Experiment and hardware	2
Inputs, tools and neural network	3-5
How reward and PPO work	6-7
Training, survival and optimizer graphs	8-10
Actual behavior and held-out evaluation	11-12
Interpretation, reproduction and reference	13-16

All performance graphs come from the saved first run. Map panels are schematic reconstructions from simulator observations. Explanatory flow diagrams are conceptual. No additional training was performed to create this report.

The experiment we actually ran

The setup follows the agreed goal: high trip score, OriginalReference, structured state, all tools and learning through trial and error.

Setting	Actual configuration
City and version	Los Angeles / OriginalReference
Environment rules	Finite materials; ordinary spawning and overcrowding failure
Training distribution	Seeds 1000-1099; 91 distinct seeds observed in 242 completed games
Held-out seeds	2000, 2001, 2002; excluded from training
Decision budget	65,536 policy selections; 35,610 completed gameplay commands
Stop rule	Decision budget or 29.5 minutes of learner time, whichever comes first
Episode cap	3,600 simulated seconds; all completed games failed before the cap
Device	Intel i7-1185G7, 32 GB RAM; CPU PyTorch, two threads, one host

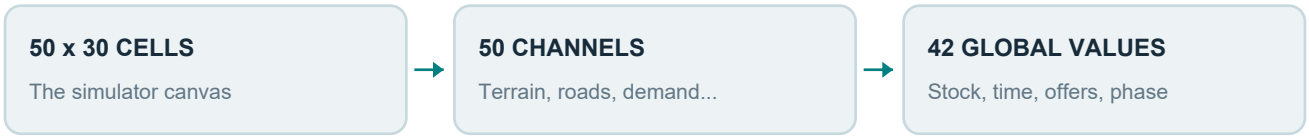


Source: progress.csv. Throughput includes simulator interaction and optimizer work; it is a cumulative rate, not an isolated inference benchmark.

Measured: 152.0 policy decisions/s and 82.6 completed commands/s. The sampled Python working set was about 378 MB; that is not a system-wide peak measurement. Installation, tests and evaluation are outside the 431.1-second learner time.

What the model can see

The network receives numerical arrays, not screenshots. A grid encodes where things are; a small global vector describes resources and the current selection stage.



Visible current information	How it enters the network
Terrain and construction	One-hot categories, eight connection bits and pending removals
Buildings	Kinds, colors, demand, reservations, entrance direction and overcrowding timer
Vehicles	Per-cell count, maximum speed and maximum current waiting
Motorways	Endpoints, retirement flag, other endpoint coordinates and number
Global state	Six inventories, time, week, score and two current upgrade offers
Selection state	Chosen tool, current phase and selected cell markers

Information deliberately withheld

Scheduled future spawns, building upgrade times, planned demand multipliers, random-generator diagnostics and offer-generation weights are excluded. Cells outside the current playable bounds are zeroed.

This is a documented structured-state interface. It exposes current internal quantities such as reservations and overcrowding timers. It does not claim to reproduce exactly what a human sees on screen.

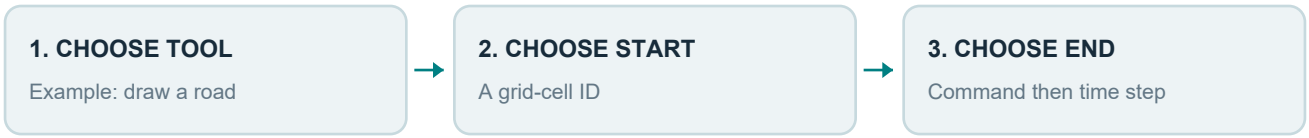
Values are normalized and clipped to [0, 1]. Variable lists are rasterized or aggregated rather than cut to a fixed list length. Some distinct entities can therefore share a cell and lose detail. The encoder is designed for Los Angeles; it does not include London-specific rail/train channels. Full channel indices appear on page 15.

What the model can do

The model chooses both the construction tool and its target cells. A route planner does not suggest house-to-destination paths.

Action family	Selections after choosing the tool
Wait / weekly upgrade 0 or 1	None; the command completes immediately
Road / bridge / light / roundabout / demolition	One target cell
Draw a road stroke	Start and end cells; the host may partially build the stroke
Change a house driveway	House cell and an adjacent direction cell
Create a motorway	Two endpoint cells
Edit a motorway	An existing endpoint, then two replacement endpoints
Remove a motorway	One of its active endpoints

A command is a bounded sequence



The vocabulary has **1,513 IDs**: 1,500 cells and 13 tools or upgrade choices. The meaning of a cell ID depends on the current phase. A motorway edit needs at most four selections; the phase sequence cannot loop forever.

Action masks filter obvious unavailable choices. For example, no motorway creation without stock, no demolition of an empty cell, and no selection outside the playable area. The simulator still decides final legality; crossing conflicts and partial strokes can be rejected or only partly succeed.

A completed or invalid command advances 20 simulation ticks, normally one simulated second. Intermediate selections advance no time and earn no reward. An impossible continuation completes a wait. Reset, step controls, stock grants and the debug reference-road command are not model choices.

Inside the neural network

This is a compact actor-critic network. The actor chooses actions; the critic estimates future reward from the current observation. Both use the same encoded city features.

GRID: 50 x 30 x 50
Channels x height x width

→

CONVOLUTION 1
16 filters; 3 x 3; stride 2

→

CONVOLUTION 2
24 filters; 3 x 3; stride 2

FLATTEN: 2,496
24 x 8 x 13 features

→

ADD 42 GLOBALS
2,538 input values

→

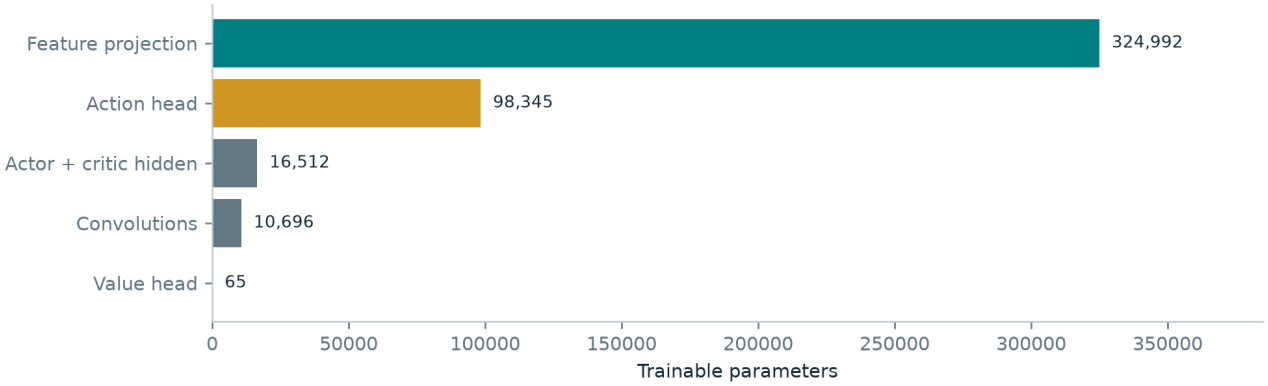
PROJECT TO 128
Linear layer + ReLU

SHARED 128-VALUE CITY FEATURES

ACTOR: 64 HIDDEN
1,513 logits -> mask -> probabilities

CRITIC: 64 HIDDEN
One expected future-return estimate

The actor and critic branch independently from the shared 128-value feature vector. ReLU is a nonlinear activation that lets layers learn more than a linear mapping.

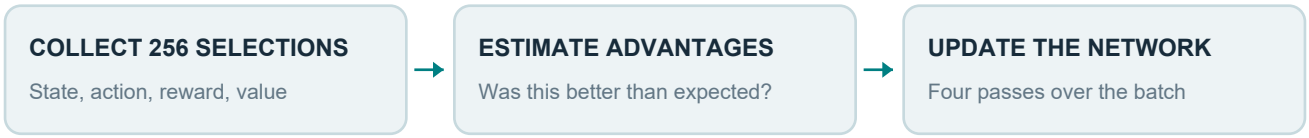


Source: final checkpoint parameter inspection. Total = 450,610 trainable parameters.

Training samples from the masked probabilities to explore. Deterministic evaluation chooses the highest-probability allowed action. There is no language model, demonstration dataset, recurrent memory or human-written routing policy inside this network.

How a trip becomes a lesson

The model starts with randomly initialized weights. It tries actions, receives a reward and changes its action probabilities based on how the observed outcome compares with its prediction.



The entire reward rule

reward = completed_trips_after - completed_trips_before
The difference spans the gameplay command plus its subsequent simulation step. There is no construction cost penalty, survival bonus or explicit failure penalty in the reward.

Example transition	Reward
Build a legal road; no trip finishes	0
Reject a road; time still advances; no trip finishes	0
Two trips finish during a completed transition	+2
Choose an intermediate endpoint; no command yet	0
City fails without another completed trip	0; episode ends

Credit arrives late

A useful road can take several selections to construct. Cars then need time to drive before a trip is counted. The learner must associate later trips with earlier decisions. Discounted returns and the critic help estimate that connection; they do not identify a correct route automatically.

Failure ends access to future trip rewards, so it can matter indirectly even without a negative terminal reward. But if almost all experiences contain zero trips, there is very little signal distinguishing one unsuccessful layout from another. This happened in the first run.

PPO is an on-policy method: new updates use recently collected interactions from the current policy. It does not maintain a persistent experience-replay dataset like some other reinforcement-learning algorithms. [R6, R7]

How PPO changes the weights

The critic estimates expected future trip reward. The actor is encouraged to favor actions with positive estimated advantage while PPO clipping limits incentives for overly large probability changes.

Advantage means an action did better or worse than the critic expected. Estimates are noisy, especially when rewards are scarce. Generalized advantage estimation combines reward prediction errors over successive selections.

```
ratio = new_probability / old_probability
policy_objective = mean(min(ratio * advantage,
clip(ratio, 0.8, 1.2) * advantage))
```

This is the core clipped policy objective, shown schematically. The implementation also uses a value-prediction loss and an entropy term. An entropy bonus encourages a broader action distribution; it does not guarantee useful exploration. Clipping is not a hard guarantee on the size of every policy change. [R6, R7]

Parameter	Run value	Practical role
Rollout length	256 selections	New interactions before an update
Minibatch / epochs	64 / 4	Four minibatches per pass, four passes per rollout
Learning rate	0.0003	Adam optimizer step scale
Discount gamma	0.999	How future selection-level reward is weighted
GAE lambda	0.95	Balances advantage-estimation trade-offs
PPO clip range	0.2	Clipped probability-ratio objective
Entropy / value weights	0.01 / 0.5	Exploration term and critic-loss weighting
Gradient norm cap	0.5	Limits gradient norm before optimizer steps

Important timing detail: gamma applies per policy selection, including intermediate endpoint selections, not per simulated second. One command can involve one to four selections. Thus command types have different selection lengths.

65,536 / 256 = 256 rollout batches. Four epochs each give 1,024 training epochs; four minibatches per epoch give 4,096 Adam steps. These counts were checked in the saved model. The last CSV row logs the previous update count (1,020), before the final four epochs complete.

Did the scores improve?

No sustained improvement is visible. Positive outcomes occurred only in episodes 9, 17, 19 and 26; every later completed training episode scored zero.



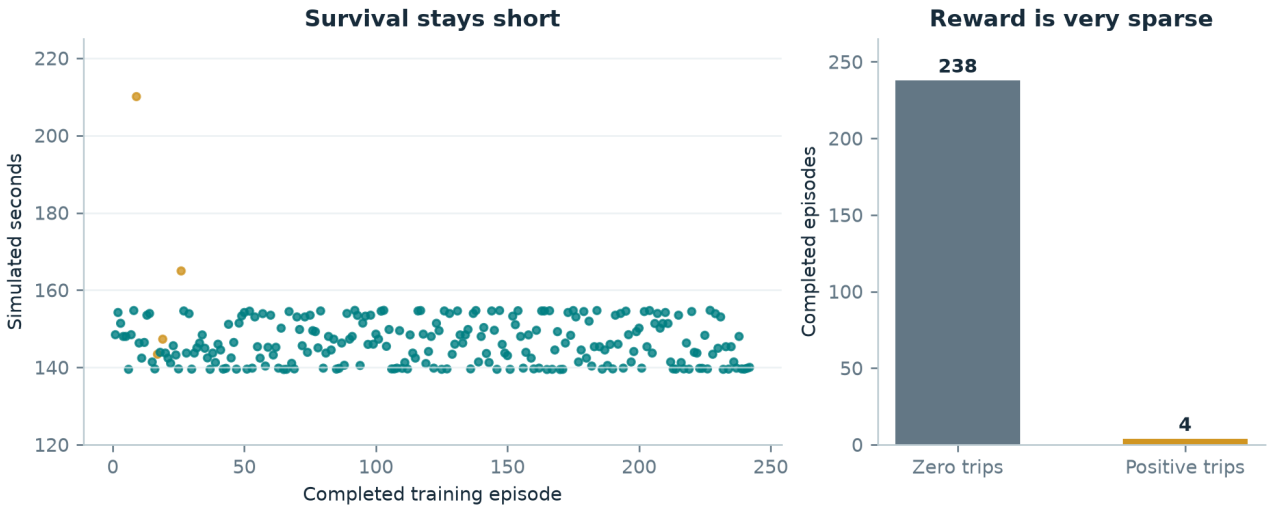
Source: episodes.jsonl, 242 completed training episodes. The score axis is deliberately zoomed to the observed 0-7 range; the goal remains 1,000 trips.

Best episode	Mean episode score	Last 30 games
7 trips at episode 9	10 total trips / 242 games = 0.0413	All scored 0

The best exploratory result is not the final policy's expected score. A single lucky network can yield trips without becoming a strategy that the model can reproduce. Training scores also come from training seeds; held-out performance is the stronger practical check.

How often did it find reward?

Only 4 of 242 completed games produced any trips: 1.65%. All 242 ended through normal game failure; none reached the external 3,600-second cap.



Source: episodes.jsonl. Gold points are games with at least one completed trip. These are observations from one training run, not independent algorithm comparisons.

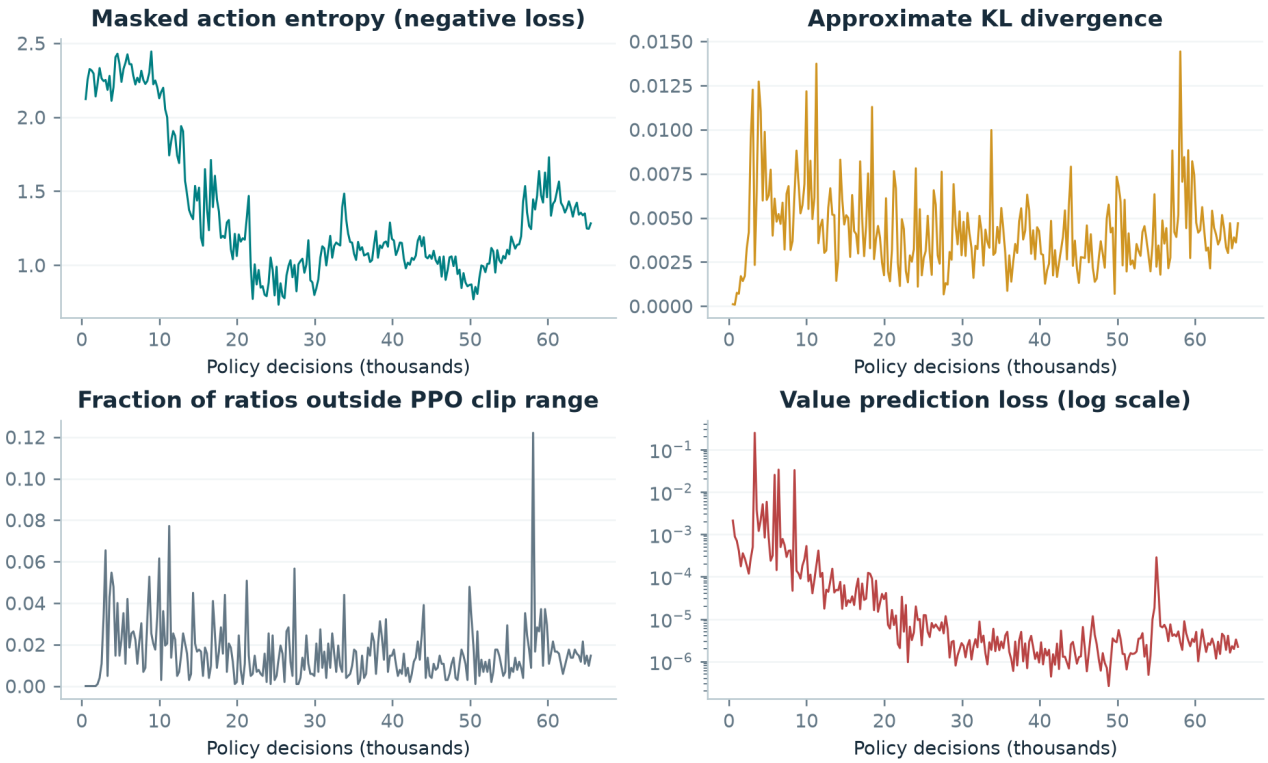
Episode	Seed	Trips
9	1020	7
17	1051	1
19	1083	1
26	1064	1

The longest completed game lasted 210.05 simulated seconds. Because even the longest episode is far below the cap, increasing that cap alone would not have helped this run survive longer.

Interpretation, not proof of a single cause: the rarity of reward supports an exploration/coordination diagnosis. The logs do not isolate whether the main cause is tool selection, spatial representation, delayed credit, optimization settings or their interaction.

The optimizer was active

These diagnostics show how the optimizer behaved. They should be read alongside score and evaluation results, not used as substitutes for game performance.



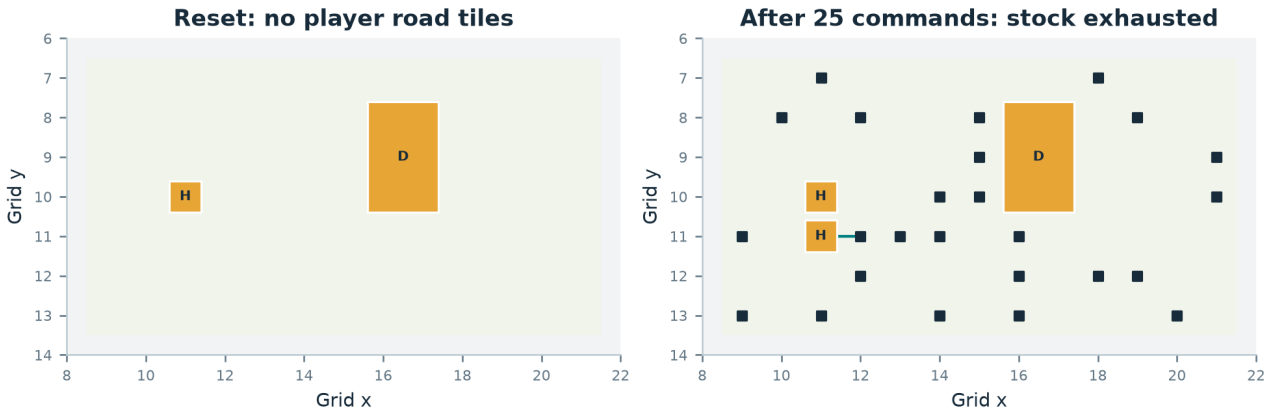
Source: progress.csv. Entropy is plotted as the negative of the logged entropy loss. Gaps at the first row reflect metrics that were not yet available.

Graph	What it can and cannot tell us
Masked action entropy	The action distribution becomes less diffuse overall. Masks and phase mix also change entropy, so this is not a clean measure of strategic exploration.
Approximate KL / clip fraction	Describe probability-distribution changes and use of the clipped objective. They do not measure correct road planning.
Value prediction loss	The critic fits its return targets. Very low loss is compatible with repeatedly predicting zero reward in zero-trip games.

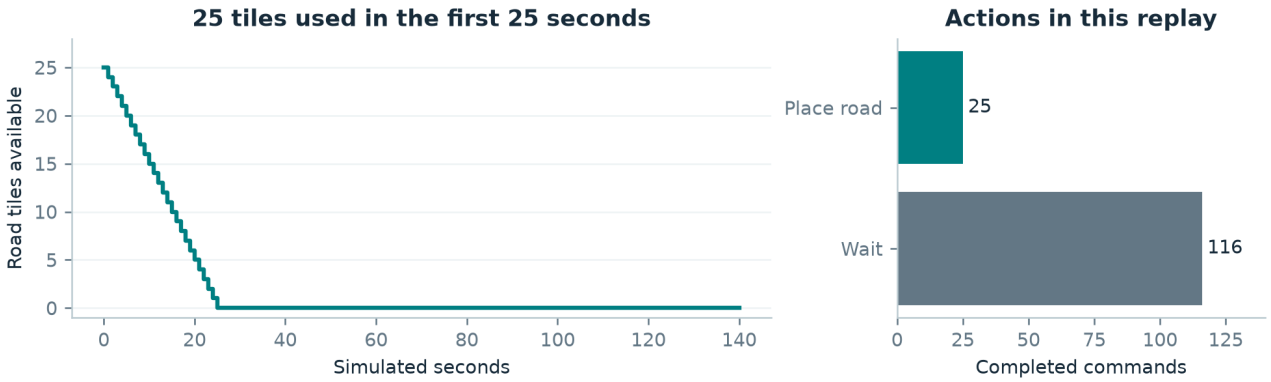
The final checkpoint has different weights from the starting checkpoint. That establishes parameter updates, not successful learning of a useful construction policy.

Watch the saved model act

A fresh-process replay of the final checkpoint, seed 2000, using highest-probability actions. This is one recorded evaluation behavior, not a claim about every possible seed or stochastic action sequence.



Schematic simulator-state maps. H = house, D = destination; colors indicate destination groups. Dark squares are player road tiles, teal lines are actual road links. Terrain is pale land, blue water and gray blocked cells. Axes are grid coordinates.



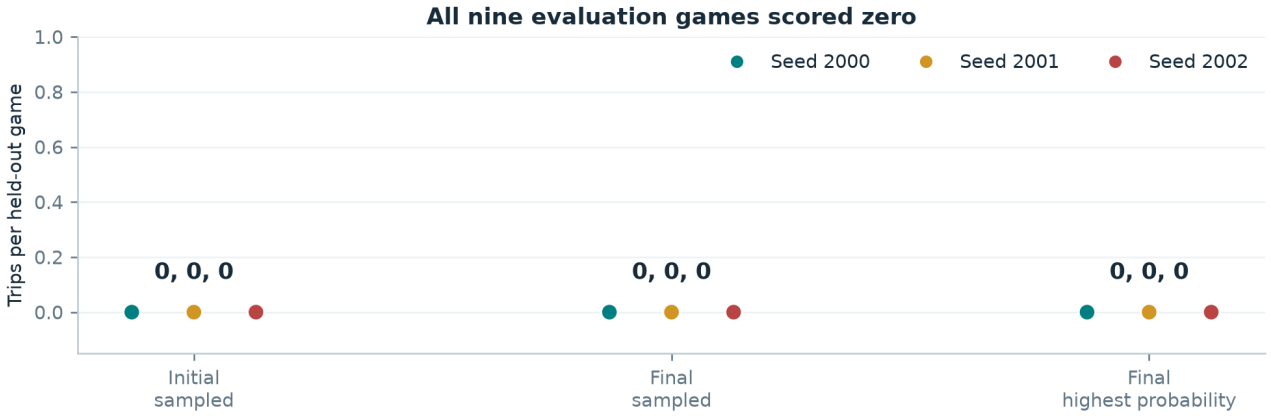
Source: replay of final.zip captured for this report. No weights were updated. Completed commands: 25 single road placements, then 116 waits.

By command 25, all starting road tiles have been spent. This behavior never chooses the draw/connect stroke. Ordinary road placement may attach a designated driveway automatically, but neighboring ordinary roads are not generally linked merely because both tiles exist.

Observed outcome: zero trips; normal game failure at 140.1 simulated seconds. The replay illustrates a concrete failure mode: spending material without constructing a productive connected route.

Test on seeds it never trained on

The same three held-out seeds were tested before learning and after loading the final checkpoint. Final evaluation used both sampled and highest-probability action selection.



Source: summary.json. Three colored points at each setting represent seeds 2000-2002. Zero values are labeled because zero-height bars would hide the result.

Seed	Initial sampled	Final sampled	Final highest-probability
2000	0	0	0
2001	0	0	0
2002	0	0	0

Every evaluation game ended through normal failure, at 139.55-140.75 simulated seconds. No evaluation reached the time cap. The final model therefore shows no held-out score improvement in this small test.

Why three seeds are a limited test

Three seeds cannot establish robust generalization across all city variations. They are sufficient to reject a claim that this particular final checkpoint demonstrated reliable high scoring here. A later model would need a broader, predeclared evaluation before any strong performance claim.

Sampled evaluation uses a fixed PyTorch seed for repeatability. The initial and final policy still produce different trajectories; using a common seed does not make the games statistically identical.

What this tells us - and what to try

The PC can execute this experiment at a practical rate. The first model has not learned the intended game strategy. More compute is not, by itself, evidence that this exact setup will reach 1,000 trips.

Evidence	Reasonable conclusion
152 decisions/s, 82.6 commands/s	The measured small CPU pipeline is feasible locally.
Reward in only 4 of 242 games	Useful construction outcomes are rare under current exploration.
7-trip best early; later scores zero	The run did not sustain or reproduce its best exploratory outcome.
Final replay spends roads, then waits	Tool and connection coordination need attention.
All held-out scores remain zero	The 1,000-trip success criterion is not met.

Proposed next experiments - not performed

1. Instrument behavior. Log action-family counts, connected house-destination pairs and material use during training. Diagnose which selections prevent trips before changing the objective.

2. Test exploration changes with the same rules. Compare initialization and entropy settings, or a reproducible progression of normal Los Angeles seeds, while retaining all tools and the held-out split. Change one factor at a time.

3. Consider spatial policy changes. A cell-wise action head or better encoding of connectivity may help coordinate endpoints. This is a hypothesis to test, not a promised improvement.

4. Agree before changing the task. Reward shaping, route suggestions, demonstrations or artificial starting states would change the agreed learning setup. They are not present in this checkpoint.

At the measured rate, one million selections would take about **1 hour 50 minutes** if throughput stayed similar. This is a compute extrapolation, not an estimate of time to a 1,000-trip policy. No longer run was started.

Reproduce and inspect the run

All training code and its documentation live on branch mohammad. Branch main remains the playground. Generated models and logs are local ignored artifacts.

File / folder	Purpose
training/environment.py	Host protocol, observation encoder, masks, staged actions and reward
training/train.py	Network, PPO configuration, time budget, checkpoints and evaluation
training/test_environment.py	Seven adapter validation checks
output/training/first-full-tools/config.json	Exact revision, packages, seeds and hyperparameters
.../episodes.jsonl and progress.csv	Episode outcomes and optimizer measurements
.../initial.zip, latest.zip, final.zip	Starting, periodic and final weights; optimizer state
.../summary.json	Before/after evaluation, throughput and parameter-change check

Run command used

```
.venv/Scripts/python.exe training/train.py
--dotnet ./tmp/dotnet/dotnet.exe
--minutes 29.5 --steps 65536 --eval-episodes 3
--output output/training/first-full-tools
```

Source revision: **30ef7ab15da7b0452e588cf171ba905ba7121459**. The run began from a clean working tree and saved copies of the training source. Packages: PyTorch 2.14.1+cpu, NumPy 2.5.4, Gymnasium 1.4.0, Stable-Baselines3 2.9.2 and SB3-Contrib 2.9.2.

What was verified

Seven adapter checks passed, including information filtering, reward timing, positive scripted trip rewards, tool command decoding and exact short replays across five maps. The existing 37 simulator suites and ten version/map protocol checks also passed. Checkpoints reloaded both during evaluation and in a separate process.

Decoder fixtures are not proof of arbitrary long learned tool sequences. Simulator tests do not certify parity with the commercial game. Normal OriginalReference fidelity remains unfinished; this model has not been tested against the commercial game.

Reference: the exact input layout

This appendix documents the feature interface used by this checkpoint. It is useful when comparing later models or checking whether hidden information leaked into an input.

Grid channels	Encoding
0	Playable-cell visibility
1-3	Terrain categories
4-8	Infrastructure categories
9-11	Building-kind categories
12-19	Building-color categories
20-27	Eight road-connection direction bits
28	Pending removal
29-31	Demand / 20; reservations / 20; overcrowding timer / 50
32-39	Eight driveway / entrance directions
40-42	Vehicle count / 10; maximum speed / 2; maximum wait / 60
43-47	Motorway endpoint; pending retirement; other endpoint x / 50 and y / 30; number / 20
48-49	Selected first cell; selected existing motorway endpoint

Global indices	Encoding
0-5	Six inventories: road stock / 100, others / 10
6-8	Elapsed time / cap; week / 100; trips / 1000
9-24	Two current offers: six type indicators, amount / 100, roads / 100 each
25-28	Four selection-phase indicators
29-41	Thirteen selected-tool indicators

All values are clipped to [0, 1], which can lose magnitude information at high counts. Shared cells and aggregate vehicle statistics can lose entity detail. The raw future schedule, RNG diagnostics and planned building changes are not passed through the encoder. Unknown color IDs, changed grid dimensions or more than two offers raise an error instead of silently truncating the schema.

Glossary and evidence sources

Use the source labels below to distinguish measurements from implementation details and general algorithm explanations.

Term	Meaning in this experiment
Policy / actor	The network distribution used to choose allowed actions
Critic / value	A prediction of discounted future trip reward
Episode	One game from reset until failure or the external cap
Policy selection	One tool or cell choice; not always a completed command
Rollout	A batch of 256 recent policy selections
Advantage	Estimated outcome relative to what the critic expected
Mask	A filter over IDs based on phase, stock and local legality
Checkpoint	Saved model and optimizer weights; not a live saved city
Held-out seed	A reserved city scenario seed not sampled for training

Measured and implementation evidence

[R1] first-full-tools/summary.json and config.json: experiment contract, before/after evaluation and throughput.
[R2] episodes.jsonl: all 242 completed training games. progress.csv: 256 logged rollout rows; last optimizer metrics precede the final update.
[R3] Saved final.zip: architecture, 450,610 parameters, 1,024 training epochs and 4,096 Adam steps verified by inspection.
[R4] Fresh deterministic replay of seed 2000 for this report: action trace and simulator snapshots; no training.
[R5] training/environment.py, training/train.py, training/test_environment.py and simulation/Headless/README.md at the recorded source revision.

Primary algorithm documentation

[R6] Stable-Baselines3 PPO documentation: stable-baselines3.readthedocs.io / PPO
[R7] OpenAI Spinning Up, PPO: spinningup.openai.com / PPO
[R8] SB3-Contrib Maskable PPO: sb3-contrib.readthedocs.io / Maskable PPO

Documentation checked on 28 October 2026. All graph data is local to the measured run or the explicitly identified checkpoint replay. Hypotheses about future improvements are proposals, not results.